

CSS Variables: var() and Custom Properties

Complete Guide: From Basics to Advanced

Contents

1. Introduction	
(a) What are CSS Variables?	2
(b) Why Use CSS Variables?	2
2. CSS Variables Basics	3
(a) Syntax and Declaration	3
i. Variable Naming	3
ii. Examples	3
(b) Using var() Function	4
i. Basic Usage	4
ii. Examples	4
(c) Scope and Inheritance	5
i. Global Scope	5
ii. Local Scope	5
3. Advanced Variable Techniques	6
(a) Fallback Values	6
i. Syntax and Examples	6
ii. Multiple Fallbacks	7
(b) Variable Composition	7
i. Combining Variables	7
ii. Examples	8
(c) Variables in calc()	8
i. Math with Custom Properties	8
ii. Examples	8
4. Theme Implementation	9
(a) Light and Dark Themes	9
i. Theme Switching	9
ii. Examples	9
(b) Brand Color Systems	10

i. Color Variable Organization	10
ii. Examples	11
5. Real-World Use Cases	11
(a) Use Case 1: Design System	11
(b) Use Case 2: Responsive Design	12
(c) Use Case 3: Dynamic Color Schemes	13
(d) Use Case 4: Maintainable Spacing	13
6. JavaScript Interaction	14
(a) Reading Variables	14
(b) Setting Variables	14
(c) Dynamic Updates	15
7. Best Practices	15
(a) Naming Conventions	15
(b) Organization	15
(c) Performance	16
8. Advanced Patterns	16
(a) Scoped Variables	16
(b) Media Query Variables	17
(c) Conditional Variables	17
9. Common Mistakes & Solutions	17
(a) Mistake 1: Invalid Variable Names	17
(b) Mistake 2: Missing Fallback	18
(c) Mistake 3: Scope Confusion	18
10. Browser Support & Compatibility	18
(a) Modern Browser Support	18
(b) Vendor Prefixes	19
11. Summary: Key Takeaways	19
12. Conclusion	20

1 Introduction

CSS Variables (also called CSS Custom Properties) allow you to store values that can be reused throughout your stylesheets. They make CSS more maintainable, DRY (Don't Repeat Yourself), and dynamic.

1.1 What are CSS Variables?

CSS Variables are named values defined with `--` prefix that can be reused anywhere in your CSS using the `var()` function.

Key characteristics:

- Start with `--` (double dash)
- Can store colors, sizes, fonts, and any CSS value
- Reusable throughout stylesheets
- Support inheritance and scoping
- Can be changed dynamically with JavaScript
- Supported in all modern browsers

1.2 Why Use CSS Variables?

CSS Variables solve common problems in stylesheet maintenance:

Problem	Without Variables	With Variables
Color changes	Find & replace	Single variable
Font consistency	Repeat in each rule	One <code>--font-family</code>
Spacing system	Manual calculation	Spacing unit variable
Theme switching	Multiple stylesheets	<code>var()</code> with JavaScript
Maintenance	Error-prone	DRY principle

2 CSS Variables Basics

2.1 Syntax and Declaration

CSS Variables are declared like regular CSS properties but start with `--`.

2.1.1 Variable Naming

```
:root {
  --primary-color: #007BFF;
  --font-size-base: 16px;
  --spacing-unit: 1rem;
  --border-radius: 4px;
  --font-family-main: Arial, sans-serif;
}
```

Naming conventions:

- Always start with -- (double dash)
- Use kebab-case (hyphens between words)
- Be descriptive: --primary-color, not --pc
- Group related variables with prefixes
- Use lowercase for consistency

Valid variable names:

```
--color: red;           /* Valid */
--font-size: 14px;      /* Valid */
--_private: blue;       /* Valid (underscore) */
--CONSTANT: 100%;       /* Valid but unconventional */

--invalid color: red;    /* Invalid (space) */
--123invalid: blue;     /* Invalid (starts with number) */
```

2.1.2 Examples

Example 1: Basic Variable Declaration

```
<!DOCTYPE html>
<html>
<head>
  <style>
    :root {
      --primary-color: #007BFF;
      --secondary-color: #6c757d;
      --success-color: #28a745;
      --font-size: 16px;
      --border-radius: 4px;
    }

    button {
      background: var(--primary-color);
      border-radius: var(--border-radius);
      font-size: var(--font-size);
    }

    button.secondary {
      background: var(--secondary-color);
    }
  </style>
</head>
<body>
  <button>Primary Button</button>
  <button class="secondary">Secondary</button>
</body>
</html>
```

Example 2: Component Variables

```
<!DOCTYPE html>
<html>
<head>
  <style>
    .card {
      --card-bg: white;
      --card-padding: 20px;
      --card-shadow: 0 2px 4px rgba(0,0,0,0.1);

      background: var(--card-bg);
      padding: var(--card-padding);
      box-shadow: var(--card-shadow);
      border-radius: 8px;
    }
  </style>
</head>
<body>
  <div class="card">
    <h3>Card Title</h3>
    <p>Card content</p>
  </div>
</body>
</html>
```

2.2 Using var() Function

The `var()` function retrieves the value of a CSS variable.

2.2.1 Basic Usage

```
:root {
  --primary-color: #007BFF;
}

button {
  background-color: var(--primary-color);
  color: var(--primary-color); /* Reuse same variable */
}
```

Syntax:

```
var(--variable-name)
var(--variable-name, fallback-value)
```

2.2.2 Examples

Example 1: Simple Variable Usage

```
<!DOCTYPE html>
<html>
```

```
<head>
  <style>
    :root {
      --main-bg: #f8f9fa;
      --text-color: #333;
      --link-color: #007BFF;
    }

    body {
      background: var(--main-bg);
      color: var(--text-color);
    }

    a {
      color: var(--link-color);
    }

    a:hover {
      color: darken(var(--link-color), 10%);
    }
  </style>
</head>
<body>
  <p>This text uses <a href="#">CSS variables</a></p>
</body>
</html>
```

Example 2: Multiple Property Assignment

```
<!DOCTYPE html>
<html>
<head>
  <style>
    :root {
      --spacing: 1rem;
    }

    .box {
      margin: var(--spacing);
      padding: var(--spacing);
      gap: var(--spacing);
    }
  </style>
</head>
<body>
  <div class="box">Content with consistent spacing</div>
</body>
</html>
```

2.3 Scope and Inheritance

Variables inherit like regular CSS properties and have scopes.

2.3.1 Global Scope

```
:root {  
    --global-color: blue;  
}  
  
/* Available everywhere */  
body { color: var(--global-color); }  
h1 { color: var(--global-color); }  
button { background: var(--global-color); }
```

Global variables defined on `:root` are available to all elements.

2.3.2 Local Scope

```
.component {  
    --component-color: green;  
    color: var(--component-color);  
}  
  
.component-child {  
    /* Inherits from parent */  
    background: var(--component-color);  
}  
  
body {  
    /* Not available here */  
    color: var(--component-color); /* Won't work */  
}
```

Local variables are only available within their scope and children.

3 Advanced Variable Techniques

3.1 Fallback Values

Fallbacks provide default values when variables aren't defined.

3.1.1 Syntax and Examples

```
/* If --primary-color is undefined, use blue */
button {
  background: var(--primary-color, blue);
}

/* With multiple properties */
.box {
  width: var(--box-width, 300px);
  height: var(--box-height, 200px);
}
```

Example 1: Basic Fallback

```
<!DOCTYPE html>
<html>
<head>
  <style>
    :root {
      --primary-color: #007BFF;
      /* --secondary-color is not defined */
    }

    button.primary {
      background: var(--primary-color, blue);
    }

    button.secondary {
      /* Uses fallback since --secondary-color
      is undefined */
      background: var(--secondary-color, gray);
    }
  </style>
</head>
<body>
  <button class="primary">Primary</button>
  <button class="secondary">Secondary</button>
</body>
</html>
```

3.1.2 Multiple Fallbacks

```
/* First undefined? Try second fallback */
color: var(--custom-color, var(--theme-color, blue));

/* Try first variable, then second, then fallback */
```



```
background:
  var(--user-bg,
    var(--theme-bg,
      white)
    );
```

3.2 Variable Composition

Variables can be combined and reused together.

3.2.1 Combining Variables

```
:root {
  --prefix: component;
  --color: blue;
  --size: 16px;
}

/* Combine variables for complex values */
.element {
  border: 1px solid var(--color);
  font: var(--size) Arial;
  background: var(--color);
}
```

3.2.2 Examples

Example 1: Shadow Composition

```
<!DOCTYPE html>
<html>
<head>
  <style>
    :root {
      --shadow-x: 0;
      --shadow-y: 2px;
      --shadow-blur: 4px;
      --shadow-color: rgba(0,0,0,0.1);
    }

    .card {
      box-shadow:
        var(--shadow-x)
        var(--shadow-y)
        var(--shadow-blur)
        var(--shadow-color);
    }
  </style>
</head>
<body>
  <div class="card">Composed shadow</div>
```

```
</body>
</html>
```

Example 2: Spacing System

```
<!DOCTYPE html>
<html>
<head>
  <style>
    :root {
      --spacing-unit: 1rem;
      --spacing-xs: calc(var(--spacing-unit) * 0.25);
      --spacing-sm: calc(var(--spacing-unit) * 0.5);
      --spacing-md: calc(var(--spacing-unit) * 1);
      --spacing-lg: calc(var(--spacing-unit) * 1.5);
      --spacing-xl: calc(var(--spacing-unit) * 2);
    }

    .box {
      padding: var(--spacing-md);
      margin: var(--spacing-lg);
    }
  </style>
</head>
</html>
```

3.3 Variables in calc()

Variables work seamlessly with CSS calc() function.

3.3.1 Math with Custom Properties

```
:root {
  --base-size: 16px;
  --multiplier: 1.5;
}

h1 {
  font-size: calc(var(--base-size) * var(--multiplier));
}

.box {
  width: calc(100% - var(--base-size) * 2);
}
```

3.3.2 Examples

Example 1: Responsive Grid

```
<!DOCTYPE html>
<html>
```

```
<head>
  <style>
    :root {
      --grid-gap: 1rem;
      --grid-cols: 3;
    }

    .grid {
      display: grid;
      grid-template-columns:
        repeat(var(--grid-cols),
          1fr);
      gap: var(--grid-gap);
    }

    @media (max-width: 768px) {
      :root {
        --grid-cols: 2;
        --grid-gap: 0.75rem;
      }
    }
  </style>
</head>
<body>
  <div class="grid">
    <div>Item 1</div>
    <div>Item 2</div>
    <div>Item 3</div>
  </div>
</body>
</html>
```

Example 2: Responsive Font Size

```
<!DOCTYPE html>
<html>
<head>
  <style>
    :root {
      --font-min: 14px;
      --font-max: 24px;
      --vw: 1vw;
    }

    h1 {
      font-size:
        clamp(
          var(--font-min),
          calc(var(--font-min) + var(--vw) * 2),
          var(--font-max)
        );
    }
  </style>
</head>
<body>
```

```
    <h1>Fluid Typography</h1>  
</body>  
</html>
```

4 Theme Implementation

4.1 Light and Dark Themes

Implement theme switching using CSS variables.

4.1.1 Theme Switching

```
:root {
  --bg-color: #ffffff;
  --text-color: #000000;
  --border-color: #cccccc;
}

body.dark-theme {
  --bg-color: #1a1a1a;
  --text-color: #ffffff;
  --border-color: #444444;
}

body {
  background: var(--bg-color);
  color: var(--text-color);
  transition: background 0.3s, color 0.3s;
}

button {
  border: 1px solid var(--border-color);
}
```

4.1.2 Examples

Example 1: Dark Mode Toggle

```
<!DOCTYPE html>
<html>
<head>
  <style>
    :root {
      --bg: white;
      --text: black;
      --card-bg: #f5f5f5;
      --shadow: rgba(0,0,0,0.1);
    }

    body.dark-theme {
      --bg: #1a1a1a;
      --text: white;
      --card-bg: #2a2a2a;
      --shadow: rgba(0,0,0,0.5);
    }

    body {
```

```
        background: var(--bg);
        color: var(--text);
        transition: all 0.3s ease;
    }

    .card {
        background: var(--card-bg);
        box-shadow: 0 2px 8px var(--shadow);
        padding: 20px;
        border-radius: 8px;
    }

    .toggle-btn {
        padding: 10px 20px;
        cursor: pointer;
    }
</style>
</head>
<body>
    <button class="toggle-btn"
        onclick="toggleTheme()">
        Toggle Theme
    </button>

    <div class="card">
        <h2>Theme-aware Card</h2>
        <p>Changes with theme</p>
    </div>

    <script>
        function toggleTheme() {
            document.body.classList.toggle(
                'dark-theme'
            );
        }
    </script>
</body>
</html>
```

4.2 Brand Color Systems

Organize colors using hierarchical variables.

4.2.1 Color Variable Organization

```
:root {
    /* Primary colors */
    --color-primary: #007BFF;
    --color-primary-dark: #0056b3;
    --color-primary-light: #0d6efd;

    /* Semantic colors */
```

```
--color-success: #28a745;
--color-warning: #ffc107;
--color-error: #dc3545;
--color-info: #17a2b8;

/* Neutral colors */
--color-white: #ffffff;
--color-gray-light: #f8f9fa;
--color-gray-medium: #6c757d;
--color-gray-dark: #212529;
}
```

4.2.2 Examples

Example 1: Complete Color System

```
<!DOCTYPE html>
<html>
<head>
  <style>
    :root {
      --primary: #007BFF;
      --success: #28a745;
      --warning: #ffc107;
      --danger: #dc3545;
      --info: #17a2b8;
    }

    .btn-primary {
      background: var(--primary);
      color: white;
    }

    .btn-success {
      background: var(--success);
      color: white;
    }

    .btn-warning {
      background: var(--warning);
      color: black;
    }

    .btn-danger {
      background: var(--danger);
      color: white;
    }

    .btn-info {
      background: var(--info);
      color: white;
    }

    button {
```

```
        padding: 10px 20px;
        border: none;
        border-radius: 4px;
        cursor: pointer;
        font-weight: bold;
    }
</style>
</head>
<body>
    <button class="btn-primary">Primary</button>
    <button class="btn-success">Success</button>
    <button class="btn-warning">Warning</button>
    <button class="btn-danger">Danger</button>
    <button class="btn-info">Info</button>
</body>
</html>
```

5 Real-World Use Cases

5.1 Use Case 1: Design System

Create a complete design system using variables.

```
<!DOCTYPE html>
<html>
<head>
    <style>
        :root {
            /* Typography */
            --font-family: -apple-system, BlinkMacSystemFont,
                'Segoe UI', Roboto;
            --font-size-sm: 0.875rem;
            --font-size-base: 1rem;
            --font-size-lg: 1.125rem;
            --font-size-xl: 1.5rem;

            /* Spacing */
            --space-xs: 0.25rem;
            --space-sm: 0.5rem;
            --space-md: 1rem;
            --space-lg: 1.5rem;
            --space-xl: 2rem;

            /* Colors */
            --color-primary: #007BFF;
            --color-text: #333;
            --color-bg: #fff;

            /* Borders */
            --border-radius: 4px;
            --border-width: 1px;
```



```
        /* Shadows */
        --shadow-sm: 0 1px 2px rgba(0,0,0,0.05);
        --shadow-md: 0 2px 4px rgba(0,0,0,0.1);
        --shadow-lg: 0 4px 12px rgba(0,0,0,0.15);
    }

    body {
        font-family: var(--font-family);
        font-size: var(--font-size-base);
        color: var(--color-text);
        background: var(--color-bg);
    }

    button {
        font-size: var(--font-size-base);
        padding: var(--space-sm) var(--space-md);
        background: var(--color-primary);
        border-radius: var(--border-radius);
        box-shadow: var(--shadow-md);
    }
</style>
</head>
<body>
    <button>Design System Button</button>
</body>
</html>
```

5.2 Use Case 2: Responsive Design

Use variables to manage responsive breakpoints.

```
<!DOCTYPE html>
<html>
<head>
    <style>
        :root {
            --columns: 3;
            --gap: 2rem;
            --font-size: 1rem;
        }

        .grid {
            display: grid;
            grid-template-columns:
                repeat(var(--columns), 1fr);
            gap: var(--gap);
        }

        body { font-size: var(--font-size); }

        @media (max-width: 768px) {
            :root {
                --columns: 2;
                --gap: 1rem;
            }
        }
    </style>
</head>
<body>
    <div class="grid">
        <div></div>
        <div></div>
        <div></div>
    </div>
</body>
</html>
```

```
        --font-size: 0.9rem;
    }
}

@media (max-width: 480px) {
    :root {
        --columns: 1;
        --gap: 0.5rem;
        --font-size: 0.8rem;
    }
}
</style>
</head>
<body>
    <div class="grid">
        <div>Item 1</div>
        <div>Item 2</div>
        <div>Item 3</div>
    </div>
</body>
</html>
```

5.3 Use Case 3: Dynamic Color Schemes

Change entire color scheme based on user preference.

```
<!DOCTYPE html>
<html>
<head>
    <style>
        :root {
            --primary: #007BFF;
            --accent: #FF6B6B;
            --bg: #fff;
            --text: #000;
        }

        .theme-ocean {
            --primary: #0066cc;
            --accent: #00ccff;
            --bg: #e6f2ff;
        }

        .theme-forest {
            --primary: #228B22;
            --accent: #32CD32;
            --bg: #e8f5e9;
        }

        .theme-sunset {
            --primary: #FF6B35;
            --accent: #FFD23F;
            --bg: #fff0e6;
        }
    </style>
</head>
```

```
    </style>
  </head>
</html>
```

5.4 Use Case 4: Maintainable Spacing

Use variables for consistent spacing throughout.

```
<!DOCTYPE html>
<html>
<head>
  <style>
    :root {
      --unit: 1rem;
    }

    body {
      padding: calc(var(--unit) * 2);
    }

    h1 {
      margin-bottom: calc(var(--unit) * 1.5);
    }

    p {
      margin-bottom: var(--unit);
      line-height: calc(var(--unit) * 1.6);
    }

    button {
      padding:
        calc(var(--unit) * 0.5)
        calc(var(--unit) * 1);
      margin: calc(var(--unit) * 0.5);
    }
  </style>
</head>
</html>
```

6 JavaScript Interaction

6.1 Reading Variables

Use JavaScript to read CSS variable values.

```
const root = document.documentElement;
const style = getComputedStyle(root);

// Get a variable value
const primaryColor =
  style.getPropertyValue('--primary-color');

console.log(primaryColor); // #007BFF
```

6.2 Setting Variables

Dynamically update CSS variables with JavaScript.

```
const root = document.documentElement;

// Set a variable
root.style.setProperty('--primary-color', '#FF0000');

// Read it back
const newColor =
  getComputedStyle(root)
    .getPropertyValue('--primary-color');

console.log(newColor); // #FF0000
```

6.3 Dynamic Updates

Example: Interactive Color Picker

```
<!DOCTYPE html>
<html>
<head>
  <style>
    :root {
      --primary-color: #007BFF;
    }

    body {
      background: var(--primary-color);
      transition: background 0.3s;
      padding: 40px;
    }

    input[type="color"] {
      width: 100px;
      height: 100px;
      cursor: pointer;
      border: none;
    }
  </style>
</head>
</html>
```

```
    }  
  </style>  
</head>  
<body>  
  <label>Pick a color:</label>  
  <input type="color" id="colorPicker"  
        value="#007BFF">  
  
  <script>  
    const colorPicker =  
      document.getElementById('colorPicker');  
    const root =  
      document.documentElement;  
  
    colorPicker.addEventListener(  
      'input',  
      (e) => {  
        root.style.setProperty(  
          '--primary-color',  
          e.target.value  
        );  
      }  
    );  
  </script>  
</body>  
</html>
```

7 Best Practices

7.1 Naming Conventions

- Use descriptive names: `--color-primary`, not `--c1`
- Group with prefixes: `--color-*`, `--space-*`, `--font-*`
- Use semantic names: `--success-color`, not `--green-color`
- Lowercase with hyphens: `--primary-color`, not `--PrimaryColor`
- Be consistent across project

7.2 Organization

```
:root {  
  /* Colors */  
  --color-primary: #007BFF;  
  --color-success: #28a745;  
  
  /* Typography */  
  --font-family: Arial, sans-serif;  
  --font-size-base: 1rem;
```

```
/* Spacing */
--space-unit: 1rem;
--space-md: var(--space-unit);

/* Borders */
--border-radius: 4px;

/* Shadows */
--shadow-md: 0 2px 4px rgba(0,0,0,0.1);
}
```

7.3 Performance

- Declare at `:root` for global availability
- Use local scope for component-specific variables
- Avoid excessive nesting of variable references
- Variables don't add significant performance overhead
- Use media queries to update variables efficiently

8 Advanced Patterns

8.1 Scoped Variables

Create component-specific variables that override globals.

```
:root {
  --color-primary: blue;
}

.card {
  --color-primary: green;
}

.card button {
  background: var(--color-primary); /* green */
}

button {
  background: var(--color-primary); /* blue */
}
```

8.2 Media Query Variables

Update variables at different breakpoints.

```
:root {
  --font-size-h1: 2.5rem;
  --padding: 2rem;
  --columns: 3;
}

@media (max-width: 768px) {
  :root {
    --font-size-h1: 2rem;
    --padding: 1.5rem;
    --columns: 2;
  }
}

@media (max-width: 480px) {
  :root {
    --font-size-h1: 1.5rem;
    --padding: 1rem;
    --columns: 1;
  }
}
```

8.3 Conditional Variables

Use with JavaScript to enable conditional styling.

```
<!DOCTYPE html>
<html>
<head>
  <style>
    :root {
      --is-dark: 0;
    }

    body {
      background:
        calc(
          var(--is-dark) * 0 +
          (1 - var(--is-dark)) * 255
        )
        % 256;
    }
  </style>
</head>
<body>
  <script>
    if (window.matchMedia(
      '(prefers-color-scheme: dark)'
    ).matches) {
      document.documentElement
        .style.setProperty('--is-dark', 1);
    }
  </script>
```

```
</body>
</html>
```

9 Common Mistakes & Solutions

9.1 Mistake 1: Invalid Variable Names

Problem: Variables don't work due to incorrect naming.

```
/* Incorrect: Space in name */
--primary color: blue;

/* Incorrect: Starts with number */
--1-color: blue;

/* Incorrect: Uses underscore prefix only */
-primary-color: blue;
```

Solution: Follow naming conventions with double dash.

```
/* Correct */
--primary-color: blue;
--color-primary: blue;
--_private: blue;
```

9.2 Mistake 2: Missing Fallback

Problem: Variable is undefined and no fallback provided.

```
/* Risky: No fallback */
button {
  background: var(--button-color);
}
```

Solution: Always provide fallback values.

```
/* Safe: Fallback provided */
button {
  background: var(--button-color, blue);
}
```

9.3 Mistake 3: Scope Confusion

Problem: Accessing variable outside its scope.

```
/* Problematic */
.component {
  --local-color: red;
}

.other {
  color: var(--local-color); /* Undefined! */
}
```


Solution: Use global scope for shared variables.

```
/* Fixed */
:root {
  --primary-color: blue;
}

.component {
  color: var(--primary-color);
}

.other {
  color: var(--primary-color);
}
```

10 Browser Support & Compatibility

10.1 Modern Browser Support

Browser	Chrome	Firefox	Safari	Edge	Opera
CSS Variables	Yes	Yes	Yes	Yes	Yes

All modern browsers support CSS variables. Internet Explorer does not.

10.2 Vendor Prefixes

No vendor prefixes are needed for CSS variables. Use them directly without `-webkit-`, `-moz-`, or `-ms-` prefixes.

11 Summary: Key Takeaways

1. **Declare with `--`** Variables must start with double dash
2. **Use `var()` to access** Reference variables with `var()` function
3. **Always use fallbacks** Provide default values for safety
4. **Scope matters** Global `:root` for shared, local for components
5. **Combine with `calc()`** Math operations with variables
6. **JavaScript integration** Update variables dynamically
7. **Theme switching** Perfect for dark mode and color schemes
8. **Maintainability** Single source of truth for design values

12 Conclusion

CSS Variables revolutionize how we write and maintain stylesheets. By embracing custom properties, you create maintainable, flexible, and scalable designs that adapt to any need. Combined with JavaScript, CSS variables enable dynamic theming and responsive design patterns that were previously difficult to achieve.

Master CSS variables and elevate your web design capabilities to the next level!